

Maven

Interview Preparation — Deep Dive

Concepts · Configuration · Commands · Q&A

For Java backend developers (1-3 years experience)
Crash-course revision with interview-grade depth

Table of Contents

1. What is Maven? Maven vs Gradle vs Ant
2. Standard Project Structure
3. pom.xml — The Heart of Maven
4. Project Coordinates (GAV)
5. Dependencies
6. Dependency Scopes
7. Transitive Dependencies
8. Dependency Conflict Resolution
9. Maven Build Lifecycle
10. Maven Goals
11. Maven Plugins
12. Maven Repositories
13. Multi-Module Projects
14. Inheritance & dependencyManagement
15. Properties
16. Profiles
17. Spring Boot + Maven
18. Common Maven Commands
19. Common Issues & Fixes
20. Maven vs Gradle
21. Top Q&A
22. Quick Cheat Sheet

1. What is Maven?

Maven is a **build automation and project management tool** for Java projects, created by the Apache Software Foundation in 2004.

What it does

1. Compiles your code
2. Manages dependencies (downloads JARs you need)
3. Runs tests
4. Packages your project (JAR/WAR/EAR)
5. Deploys to repositories
6. Generates reports & documentation

Core philosophy: "Convention over Configuration"

Maven enforces a standard project structure so any Maven project looks the same. You don't write build scripts from scratch — you describe **what** to build, and Maven knows **how**.

Maven vs Gradle vs Ant

Tool	Style	Build script	Popularity
Ant	Imperative — scripted XML	build.xml	Legacy
Maven	Declarative — describe project	pom.xml	Most enterprise Java
Gradle	Hybrid — Groovy/Kotlin DSL	build.gradle	Android, modern projects

Gradle is faster (incremental builds, daemon) and more flexible. Maven wins on **simplicity and convention**.

2. Standard Project Structure

Maven follows a **convention** — your code goes in specific folders:

```
my-project/
├── pom.xml                ← project descriptor (THE central file)
├── src/
│   ├── main/
│   │   ├── java/         ← production source code
│   │   ├── resources/    ← config files (.properties, .yml, .xml)
│   │   └── webapp/        ← (for WAR projects) HTML/JSP/static
│   └── test/
│       ├── java/         ← test source code
│       └── resources/    ← test resources
├── target/               ← build output (compiled .class, JARs)
└── .gitignore
```

Always include `target/` in `.gitignore` — it's regenerated.

Knowing the convention is expected. If asked "where do unit tests live?" → `src/test/java/`.

3. pom.xml — The Heart of Maven

pom.xml (Project Object Model) is an XML file that describes your project: identity, dependencies, plugins, build settings.

Minimal pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.mycompany</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
      <version>3.2.0</version>
    </dependency>
  </dependencies>
</project>
```

4. Project Coordinates (GAV)

Every Maven artifact is uniquely identified by 3 things:

Coordinate	Description	Example
groupId	Organization / namespace (reverse domain)	com.mycompany.app
artifactId	Project name	my-service
version	Version	1.0.0 or 1.0.0-SNAPSHOT

Sometimes called **GAV coordinates**. Optional 4th: **packaging** (jar/war/pom) and **classifier** (sources, javadoc).

Why this matters

- Maven Central (and all Maven repos) indexes everything by GAV
- Two artifacts with same GAV are considered the SAME artifact

Version conventions

Format	Meaning
1.0.0	Release version (immutable)
1.0.0-SNAPSHOT	Development version (mutable, re-downloaded)
1.0.0-RC1	Release candidate
1.0.0-alpha, -beta	Pre-release

SNAPSHOT vs Release — interview question

- **SNAPSHOT** versions are checked for updates on each build (in case the dependency author published a newer build)
- **Release** versions are cached forever (immutable)

5. Dependencies

Adding a dependency

```
<dependencies>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.15.0</version>
  </dependency>
</dependencies>
```

Maven downloads the JAR (and ITS dependencies — **transitive dependencies**) automatically.

Where dependencies are stored

Locally cached in: `~/.m2/repository/` — then copied/referenced when needed for builds.

6. Dependency Scopes

Scope tells Maven **WHEN** the dependency is needed.

Scope	Compile?	Test?	Runtime?	Packaged?	Example
compile (default)	✓	✓	✓	✓	App libraries (Spring, Jackson)
test	✗	✓	✗	✗	JUnit, Mockito
runtime	✗	✓	✓	✓	JDBC drivers
provided	✓	✓	✗	✗	Servlet API (provided by Tomcat)
system	✓	✓	—	✗	Local JAR (deprecated, avoid)
import	—	—	—	—	Used in dependencyManagement for BOMs

Examples

```
<!-- compile (default) -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <version>6.1.0</version>
</dependency>

<!-- test (only for tests) -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>

<!-- runtime (driver – needed at runtime, not compile) -->
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
  <scope>runtime</scope>
</dependency>

<!-- provided (Servlet API – server provides at runtime) -->
<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
  <version>6.0.0</version>
  <scope>provided</scope>
</dependency>
```

Q: When to use provided?

For libraries the runtime container provides — Servlet API in Tomcat, EJB API in JEE servers. Don't bundle them in your WAR/JAR.

Q: When to use runtime?

For things you don't compile against but need at runtime — JDBC drivers, logging implementations like Logback.

7. Transitive Dependencies (Important!)

If your project depends on A, and A depends on B → Maven automatically pulls B too. This is **transitive dependency resolution**.

Example

```
Your app depends on: Spring Boot Web
Spring Boot Web depends on: Spring Web, Tomcat, Jackson
Spring Web depends on: Spring Core, Spring Beans
...
```

You add ONE dependency, Maven brings in 30+ JARs.

Why this matters

- Saves you specifying every transitive dep manually
- But causes **dependency conflicts** when two paths bring different versions

View dependency tree

```
mvn dependency:tree
```

Output:

```
[INFO] +- org.springframework.boot:spring-boot-starter-web:jar:3.2.0
[INFO] |   +- org.springframework.boot:spring-boot-starter:jar:3.2.0
[INFO] |   |   +- org.springframework:spring-core:jar:6.1.0
...
```

8. Dependency Conflict Resolution

When the tree contains different versions of the same dependency, Maven picks **one**.

Maven's resolution rule: "Nearest Wins"

- The version closer to your project (fewer hops) wins
- If equidistant, **first declaration wins**

Example

```
Your app
├─ A → uses Jackson 2.15.0
└─ B → uses Jackson 2.13.0
```

Both are 1 hop away — first one declared wins.

Force a specific version — `<dependencyManagement>`

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.15.0</version>      <!-- forces this version everywhere -->
    </dependency>
  </dependencies>
</dependencyManagement>
```

Exclude a transitive dependency

```
<dependency>
  <groupId>com.example</groupId>
  <artifactId>some-lib</artifactId>
  <version>1.0</version>
  <exclusions>
    <exclusion>
      <groupId>org.unwanted</groupId>
      <artifactId>bad-lib</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

Q: Two libraries pull different Jackson versions — how do you resolve?

1. `mvn dependency:tree` to identify. 2. Use `<dependencyManagement>` to force one version. 3. Or `<exclusions>` to remove the unwanted transitive.

9. Maven Build Lifecycle

Maven defines **3 built-in lifecycles**:

Lifecycle	Purpose
default	Build & deploy
clean	Remove build artifacts
site	Generate project documentation

Default Lifecycle Phases (in order)

You'll most often work with these. Each phase invokes the previous ones first.

Phase	What happens
validate	Check project structure is valid
compile	Compile source code (src/main/java)
test-compile	Compile test code (src/test/java)
test	Run unit tests (Surefire plugin)
package	Bundle into JAR/WAR (in target/)
verify	Run integration tests, quality checks
install	Copy artifact to local ~/.m2 repo
deploy	Upload to remote repo (Nexus, Artifactory)

Important: phases run cumulatively. Running `mvn install` automatically runs: validate → compile → test → package → verify → install.

Clean Lifecycle

pre-clean → clean → post-clean

Common combos

```
mvn clean install      # most common – clean + full build + install
mvn clean package      # build but don't install
mvn clean verify       # run all tests but don't install
```

Q: Difference between mvn package and mvn install?

`package` builds the JAR/WAR in `target/`. `install` does that AND copies it to your local Maven repo (`~/.m2`) so other local projects can depend on it.

Q: Difference between install and deploy?

`install` → local `~/.m2`. `deploy` → remote repository (Nexus, Artifactory) for sharing across teams.

10. Maven Goals

Each phase is implemented by **goals** of plugins. Format: `pluginName:goalName`

Examples

```
mvn compiler:compile      # the compile goal of the compiler plugin
mvn surefire:test         # run tests
mvn jar:jar               # build a JAR
mvn dependency:tree       # show dependency tree
mvn dependency:analyze    # find unused/undeclared deps
mvn help:effective-pom    # show fully-resolved POM (with inheritance)
```

You can run goals directly without going through the lifecycle.

Common useful goals

```
mvn dependency:tree       # see all dependencies
mvn dependency:analyze    # find unused deps
mvn dependency:purge-local-repository # rebuild local cache
mvn versions:display-dependency-updates # check for newer versions
mvn help:effective-pom    # see resolved POM (parent + this)
mvn compile              # just compile
mvn test                 # just run tests
```

11. Maven Plugins

Plugins do the actual work. Maven's "core" is tiny — everything is a plugin.

Common plugins (mostly auto-configured)

Plugin	Purpose
maven-compiler-plugin	Compiles Java source
maven-surefire-plugin	Runs unit tests (*Test.java)
maven-failsafe-plugin	Runs integration tests (*IT.java)
maven-jar-plugin	Builds JARs
maven-war-plugin	Builds WARs
maven-shade-plugin	Builds "fat JAR" (with all deps)
maven-assembly-plugin	Custom packaging (zip, tar)
maven-dependency-plugin	Dependency utilities
spring-boot-maven-plugin	Builds executable Spring Boot JARs
jacoco-maven-plugin	Code coverage
maven-release-plugin	Automate release process

Configuring a plugin in pom.xml

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>17</source>
        <target>17</target>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Spring Boot plugin example (executable JAR)

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Now `mvn package` produces an executable JAR runnable with `java -jar`.

12. Maven Repositories

Maven downloads dependencies from repositories — directories of JARs.

Three types

Type	Location	Purpose
Local	~/.m2/repository/	Your machine's cache
Central	https://repo.maven.apache.org/maven2/	Public, default
Remote/Private	Nexus, Artifactory	Company-internal artifacts

How resolution works

1. Maven checks **local** repo first
2. If not found → downloads from **remote** repos (Central by default)
3. Caches in local for next time

Adding a private repository

```
<repositories>
  <repository>
    <id>company-nexus</id>
    <url>https://nexus.mycompany.com/repository/maven-public/</url>
  </repository>
</repositories>
```

settings.xml (~/.m2/settings.xml)

Configures global Maven behavior — credentials for private repos, proxies, mirror servers.

```
<settings>
  <servers>
    <server>
      <id>company-nexus</id>
      <username>myuser</username>
      <password>${env.NEXUS_PASSWORD}</password>
    </server>
  </servers>
  <mirrors>
    <mirror>
      <id>internal-mirror</id>
      <mirrorOf>central</mirrorOf>
      <url>https://nexus.mycompany.com/repository/maven-central/</url>
    </mirror>
  </mirrors>
</settings>
```

13. Multi-Module Projects

For larger projects, you split code into multiple modules. One **parent POM** + child POMs.

Structure

```
parent-project/
├── pom.xml           ← parent POM
├── module-common/
│   └── pom.xml
├── module-service/
│   └── pom.xml
└── module-web/
    └── pom.xml
```

Parent pom.xml

```
<project>
  <groupId>com.mycompany</groupId>
  <artifactId>parent-project</artifactId>
  <version>1.0.0</version>
  <packaging>pom</packaging>          <!-- IMPORTANT: pom for parent -->

  <modules>
    <module>module-common</module>
    <module>module-service</module>
    <module>module-web</module>
  </modules>

  <dependencyManagement>
    <!-- versions defined here, inherited by children -->
  </dependencyManagement>
</project>
```

Child pom.xml

```
<project>
  <parent>
    <groupId>com.mycompany</groupId>
    <artifactId>parent-project</artifactId>
    <version>1.0.0</version>
  </parent>

  <artifactId>module-service</artifactId>

  <dependencies>
    <dependency>
      <groupId>com.mycompany</groupId>
      <artifactId>module-common</artifactId>
    </dependency>
  </dependencies>
</project>
```

Benefits

- Centralized version management
- Shared plugin config
- Build all modules in correct order: `mvn clean install` from parent

14. Inheritance & dependencyManagement vs dependencies

<dependencies> — actually included

Children that declare a dependency listed under parent's `<dependencies>` automatically get it.

<dependencyManagement> — version control only

Children must explicitly declare the dependency, but **don't need to specify version** — it comes from parent.

```
<!-- Parent -->
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.15.0</version>
    </dependency>
  </dependencies>
</dependencyManagement>

<!-- Child -->
<dependencies>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <!-- no version – inherited -->
  </dependency>
</dependencies>
```

Q: Difference between <dependencies> and <dependencyManagement>?

`<dependencies>` actually adds the dependency. `<dependencyManagement>` just centralizes version metadata; child must still declare to use it.

15. Properties

Reusable variables in pom.xml.

```
<properties>
  <java.version>17</java.version>
  <spring.version>6.1.0</spring.version>
  <maven.compiler.source>${java.version}</maven.compiler.source>
  <maven.compiler.target>${java.version}</maven.compiler.target>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <version>${spring.version}</version>
  </dependency>
</dependencies>
```

Built-in properties

- `${project.version}` — current project's version
- `${project.basedir}` — project root
- `${env.JAVA_HOME}` — environment variables
- `${user.home}` — system properties

16. Profiles

Configurations that activate under specific conditions — like environment-specific builds.

```
<profiles>
  <profile>
    <id>dev</id>
    <activation>
      <activeByDefault>true</activeByDefault>
    </activation>
    <properties>
      <db.url>jdbc:h2:mem:testdb</db.url>
    </properties>
  </profile>

  <profile>
    <id>prod</id>
    <properties>
      <db.url>jdbc:postgresql://prod-db:5432/myapp</db.url>
    </properties>
  </profile>
</profiles>
```

Activate a profile

```
mvn package -P prod           # explicit
mvn package -Pdev,debug       # multiple
```

Or auto-activate based on JDK version, OS, or property:

```
<activation>
  <jdk>17</jdk>                                <!-- when JDK 17 -->
  <os><family>Windows</family></os>
  <property><name>environment</name><value>prod</value></property>
</activation>
```

17. Spring Boot + Maven (Real-World)

Spring Boot starter parent — common pattern

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.0</version>
</parent>
```

The `spring-boot-starter-parent` provides:

- Sensible default plugin configurations
- Curated dependency versions (BOM — Bill of Materials)
- Java version defaults
- Resource filtering for application.properties

Add starters (curated dep groups)

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <!-- no version needed - parent BOM provides -->
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Build executable JAR

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

```
mvn clean package
java -jar target/myapp-1.0.0-SNAPSHOT.jar
```

18. Common Maven Commands (Cheat Sheet)

```
# Lifecycle
mvn validate           # check project
mvn compile            # compile main code
mvn test               # run tests
mvn package            # create JAR/WAR
mvn install            # install to local repo
mvn deploy             # push to remote repo

# Combos (most used)
mvn clean install      # clean + full build + install
mvn clean package      # clean + build (no install)
mvn clean test         # clean + run tests
mvn clean verify       # clean + run all checks

# Skip tests
mvn install -DskipTests # skip running but compile
mvn install -Dmaven.test.skip=true # skip compiling AND running

# Run a specific test
mvn test -Dtest=UserServiceTest
mvn test -Dtest=UserServiceTest#testRegister # specific method

# Profiles
mvn package -P prod
mvn package -Pdev,debug

# Inspection
mvn dependency:tree
mvn dependency:analyze
mvn help:effective-pom
mvn versions:display-dependency-updates
mvn versions:display-plugin-updates

# Run a goal
mvn spring-boot:run    # for Spring Boot apps
mvn exec:java -Dexec.mainClass="com.myapp.Main"

# Offline mode
mvn -o clean install   # use only local repo

# Force update (snapshots)
mvn -U clean install   # check for newer SNAPSHOTs

# Verbose / Debug
mvn -X clean install   # debug output
```

19. Common Issues & Fixes

"Could not find artifact" / "Cannot resolve dependency"

- Check spelling of GAV
- Check repository availability (`mvn -U` to force update)
- Check `~/.m2/settings.xml` for repository config
- Maven Central might not have it → check on <https://search.maven.org>

Stale dependencies / weird build issues

```
rm -rf ~/.m2/repository          # nuclear option
mvn dependency:purge-local-repository # cleaner
mvn -U clean install            # force update
```

Building offline (no internet)

```
mvn -o clean install          # use only what's cached
```

Build fails because of test failure

```
mvn install -DskipTests      # skip for now (not best practice)
```

Old JDK version being used

Check `pom.xml` properties:

```
<maven.compiler.source>17</maven.compiler.source>
<maven.compiler.target>17</maven.compiler.target>
```

Or check `JAVA_HOME` env var.

Out-of-memory while building

```
export MAVEN_OPTS="-Xmx2g"
mvn clean install
```

20. Maven vs Gradle (interview comparison)

Feature	Maven	Gradle
Build script	XML (verbose)	Groovy / Kotlin DSL (concise)
Speed	Slower (no daemon)	Faster (daemon, incremental)
Convention	Strong	Flexible
Learning curve	Easier	Steeper
Customization	Limited (plugin-based)	Highly flexible
Ecosystem	Massive (older)	Growing (newer)
Default for	Spring Boot, enterprise	Android, modern projects

When to pick which

- **Maven** — standard enterprise, simple, predictable
- **Gradle** — performance, large multi-module projects, complex builds

21. Top Maven Interview Q&A

Q: What is Maven?

A build automation and project management tool for Java. It compiles, tests, packages, and deploys projects, manages dependencies, and follows the "convention over configuration" principle.

Q: Why use Maven over manually running javac?

Manages dependencies (transitive, conflict resolution); standardizes project structure; builds, tests, packages with one command; handles complex multi-module builds; integrates with IDEs and CI/CD.

Q: What is pom.xml?

The Project Object Model — central XML file describing project identity, dependencies, plugins, build settings.

Q: What are GAV coordinates?

groupId + artifactId + version — uniquely identify a Maven artifact.

Q: SNAPSHOT vs Release version?

SNAPSHOT (e.g., 1.0.0-SNAPSHOT) — development version, mutable, Maven re-checks for updates. Release (1.0.0) — final version, immutable, cached forever.

Q: What's a transitive dependency?

A dependency of your dependency. When you add A, and A depends on B, Maven automatically pulls B too.

Q: How does Maven resolve dependency conflicts?

"Nearest wins" — closer dependency in the tree wins. If equidistant, first declared wins. Override via `<dependencyManagement>` or `<exclusions>`.

Q: How to view the dependency tree?

```
mvn dependency:tree
```

Q: How to find unused or missing dependencies?

```
mvn dependency:analyze
```

Q: Common dependency scopes?

`compile` (default) — used everywhere. `test` — only for tests (JUnit). `runtime` — needed at runtime, not at compile (JDBC drivers). `provided` — provided by container (Servlet API, Tomcat).

Q: Difference between compile and provided scope?

Both compile-time available. `compile` packaged in JAR/WAR. `provided` is NOT packaged — assumed runtime container provides it.

Q: What is the Maven build lifecycle?

A defined sequence of phases: validate → compile → test → package → verify → install → deploy. Each phase invokes the previous ones.

Q: Difference between mvn package and mvn install?

`package` builds JAR/WAR in `target/`. `install` does that + copies to local `~/.m2` so other local projects can use it.

Q: Difference between install and deploy?

`install` → local `~/.m2`. `deploy` → remote repo (Nexus/Artifactory) for team sharing.

Q: How to skip tests?

`-DskipTests` — skip running tests (still compile). `-Dmaven.test.skip=true` — skip both compiling AND running.

Q: What's a Maven plugin?

A unit that provides goals (work units). E.g., compiler plugin compiles code, surefire runs tests. Maven core is small; plugins do the work.

Q: What's the difference between phase and goal?

Phase is a stage in the lifecycle (compile, test). **Goal** is a specific task in a plugin (compiler:compile). Phases bind to goals — running a phase executes the goals bound to it.

Q: Three types of Maven repositories?

Local (`~/.m2/repository/`), Central (Maven Central), Remote/Private (Nexus, Artifactory).

Q: What is <dependencyManagement>?

Section that centralizes version management without actually adding dependencies. Children declare deps without versions; versions inherited from parent.

Q: What's a multi-module project?

A parent POM containing multiple sub-modules. Common for large apps (one repo, multiple deployable units). Parent has `<packaging>pom</packaging>`.

Q: What's a profile in Maven?

A build configuration for a specific environment/condition (dev, prod, OS-specific). Activated via `-P` flag or auto-activation rules.

Q: How do you handle different configurations for dev/prod?

Maven profiles, Spring profiles (@Profile, application-dev.properties), externalized config (env vars, vault).

Q: What's a "fat JAR" / "uber JAR"?

A JAR containing your code AND all dependencies — runnable with `java -jar`. Built via `maven-shade-plugin` or `spring-boot-maven-plugin`.

Q: What's a BOM (Bill of Materials)?

A POM listing dependency versions for a project family (e.g., Spring Boot's BOM). Imported via `<dependencyManagement>` `<scope>import</scope>`. Ensures all related libs use compatible versions.

Q: Which directory contains the Maven local cache?

`~/.m2/repository/`

Q: Where do application.properties files go?

`src/main/resources/` for production, `src/test/resources/` for tests.

Q: What happens during mvn clean?

Deletes the `target/` directory — removes all build outputs. Often combined: `mvn clean install`.

Q: How to use snapshot dependencies in development?

Maven re-checks for SNAPSHOT updates daily by default. Force with `mvn -U clean install`.

Q: What's the role of settings.xml?

Global config in `~/.m2/settings.xml` — credentials, proxies, mirrors. Doesn't go in source control (contains secrets).

Q: Can you have multiple versions of the same dependency?

At classpath level — only ONE version is used (Maven picks via "nearest wins"). Multiple versions in the tree → conflict resolution applies.

Q: How does Maven know which Java version to use?

`<maven.compiler.source>` and `<target>` in pom.xml. Or `<source>` / `<target>` in compiler plugin config. `JAVA_HOME` env var must point to correct JDK.

Q: How to run a Spring Boot app via Maven?

`mvn spring-boot:run` — uses the spring-boot-maven-plugin to launch the app without packaging.

Q: What is Maven Wrapper (mvnw)?

A script (mvnw/mvnw.cmd) checked into the repo that downloads the right Maven version. Ensures reproducible builds — no need for everyone to install Maven manually. Like Gradle Wrapper.

Q: How to create a new Maven project?

`mvn archetype:generate -DgroupId=com.mycompany -DartifactId=my-app -DarchetypeArtifactId=maven-archetype-quickstart`. Or use IDE wizards / Spring Initializr (start.spring.io).

Q: How does Maven differ from npm/pip?

All are dependency managers. Maven uses XML (verbose), npm uses JSON, pip uses text. Maven has stricter conventions. Maven also handles full build lifecycle, npm focuses on JS dependencies + scripts.

22. Quick Cheat Sheet

mvn clean install	# most common – clean + full build
mvn clean package	# build, no install
mvn clean test	# run tests only
mvn dependency:tree	# see dependency graph
mvn dependency:analyze	# find unused/undeclared deps
mvn help:effective-pom	# see resolved POM
mvn -DskipTests install	# skip tests
mvn -P prod package	# use prod profile
mvn -U clean install	# force snapshot updates
mvn spring-boot:run	# run Spring Boot app

```
<!-- POM essentials -->
<groupId>com.mycompany</groupId>
<artifactId>my-app</artifactId>
<version>1.0.0-SNAPSHOT</version>
<packaging>jar</packaging>

<!-- Java version -->
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
</properties>

<!-- Dependency -->
<dependency>
  <groupId>...</groupId>
  <artifactId>...</artifactId>
  <version>...</version>
  <scope>test</scope>
</dependency>
```

End of Maven Interview Prep
Master GAV, lifecycle, scopes, and conflict resolution — you're ready.